

Colby Crutcher

Lab 3

Secure Coding

Part 1

In this section, you'll be using pwndbg to perform some static analysis of the included binary (the C source files are not included and so commands like list will not be usable).

1. Disassemble main and state the contents of main. (print statements and function calls in proper order)

```
pwndbg> disass main
Dump of assembler code for function main:
0x00000000000012a3 <+0>:    endbr64
0x00000000000012a7 <+4>:    push    rbp
0x00000000000012a8 <+5>:    mov     rbp, rsp
0x00000000000012ab <+8>:    sub     rsp, 0x20
0x00000000000012af <+12>:   mov     DWORD PTR [rbp-0x14], edi
0x00000000000012b2 <+15>:   mov     QWORD PTR [rbp-0x20], rsi
0x00000000000012b6 <+19>:   mov     rax, QWORD PTR fs:0x28
0x00000000000012bf <+28>:   mov     QWORD PTR [rbp-0x8], rax
0x00000000000012c3 <+32>:   xor     eax, eax
0x00000000000012c5 <+34>:   lea     rdi, [rip+0xda0]          # 0x206c
0x00000000000012cc <+41>:   call    0x10a0 <puts@plt>
0x00000000000012d1 <+46>:   mov     edi, 0x3
0x00000000000012d6 <+51>:   call    0x1218 <simpleCounter>
0x00000000000012db <+56>:   lea     rdi, [rip+0xda3]          # 0x2085
0x00000000000012e2 <+63>:   mov     eax, 0x0
0x00000000000012e7 <+68>:   call    0x10c0 <printf@plt>
0x00000000000012ec <+73>:   lea     rax, [rbp-0xc]
0x00000000000012f0 <+77>:   mov     rsi, rax
0x00000000000012f3 <+80>:   lea     rdi, [rip+0xd9b]          # 0x2095
0x00000000000012fa <+87>:   mov     eax, 0x0
0x00000000000012ff <+92>:   call    0x10d0 <__isoc99_scanf@plt>
0x0000000000001304 <+97>:   mov     eax, DWORD PTR [rbp-0xc]
0x0000000000001307 <+100>:  mov     edi, eax
0x0000000000001309 <+102>:  call    0x11c9 <simpleIf>
0x000000000000130e <+107>:  mov     eax, DWORD PTR [rbp-0xc]
0x0000000000001311 <+110>:  mov     edi, eax
0x0000000000001313 <+112>:  call    0x1218 <simpleCounter>
0x0000000000001318 <+117>:  mov     eax, 0x0
0x000000000000131d <+122>:  mov     rdx, QWORD PTR [rbp-0x8]
0x0000000000001321 <+126>:  xor     rdx, QWORD PTR fs:0x28
0x000000000000132a <+135>:  je      0x1331 <main+142>
0x000000000000132c <+137>:  call    0x10b0 <__stack_chk_fail@plt>
0x0000000000001331 <+142>:  leave
0x0000000000001332 <+143>:  ret
End of assembler dump.
```

4 → main at cybr437lab2main.c:9

5 → main at cybr437lab2main.c:23

6 → simpleCounter at cybr437lab2.c:22

7 → simpleIf at cybr437lab2.c:9

2. Set a breakpoint at line 23 of main and on each of the two functions being called from main.

```
pwndbg> info break
Num  Type      Disp Enb Address          What
4    breakpoint keep y  0x00000000000012b6 in main at cybr437lab2main.c:9
5    breakpoint keep y  0x0000000000001318 in main at cybr437lab2main.c:23
6    breakpoint keep y  0x0000000000001227 in simpleCounter at cybr437lab2.c:22
7    breakpoint keep y  0x00000000000011d8 in simpleIf at cybr437lab2.c:9
pwndbg>
```

3. Start running the program. What are the contents of the first string in main?

- The first string is “This is a pwndbg program”

```
0x555555552b6 <main+19> mov rax, qword ptr fs:[0x28] RAX, [0x7ffff7fb4768] => 0x30b026171dba3e00
0x555555552bf <main+28> mov qword ptr [rbp - 8], rax [0x7ffff7fdda8] => 0x30b026171dba3e00
0x555555552c3 <main+32> xor eax, eax EAX => 0
0x555555552c5 <main+34> lea rdi, [rip + 0xda0] RDI => 0x55555555606c ← 'This is a pwndbg program'
0x555555552cc <main+41> call puts@plt <puts@plt>
```

4. Continue to the next breakpoint. What is the name and the value being passed into the function called?

- This function is called simpleCounter. The value being passed into it is num = 3.

```
pwndbg> c
Continuing.
This is a pwndbg program

Breakpoint 2, simpleCounter (num=3) at cybr437lab2.c:22
```

5. What function did you just come from and what function are you stopped in?

- We are in simpleCounter, and we came from main.

6. Disassemble this function you are stopped at and capture the output

```

pwndbg> disass simpleCounter
Dump of assembler code for function simpleCounter:
0x000055555555218 <+0>:      endbr64
0x00005555555521c <+4>:      push    rbp
0x00005555555521d <+5>:      mov     rbp, rsp
0x000055555555220 <+8>:      sub     rsp, 0x20
0x000055555555224 <+12>:     mov     DWORD PTR [rbp-0x14], edi
=> 0x000055555555227 <+15>:     mov     eax, DWORD PTR [rip+0x2de3]          # 0x5555555558010 <s>
0x00005555555522d <+21>:     mov     esi, eax
0x00005555555522f <+23>:     lea     rdi, [rip+0xe02]                    # 0x5555555556038
0x000055555555236 <+30>:     mov     eax, 0x0
0x00005555555523b <+35>:     call    0x5555555550c0 <printf@plt>
0x000055555555240 <+40>:     mov     DWORD PTR [rbp-0x4], 0x0
0x000055555555247 <+47>:     jmp     0x555555555275 <simpleCounter+93>
0x000055555555249 <+49>:     mov     eax, DWORD PTR [rip+0x2dc1]          # 0x5555555558010 <s>
0x00005555555524f <+55>:     mov     esi, eax
0x000055555555251 <+57>:     lea     rdi, [rip+0xe06]                    # 0x555555555605e
0x000055555555258 <+64>:     mov     eax, 0x0
0x00005555555525d <+69>:     call    0x5555555550c0 <printf@plt>
0x000055555555262 <+74>:     mov     eax, DWORD PTR [rip+0x2da8]          # 0x5555555558010 <s>
0x000055555555268 <+80>:     add     eax, 0x1
0x00005555555526b <+83>:     mov     DWORD PTR [rip+0x2d9f], eax          # 0x5555555558010 <s>
0x000055555555271 <+89>:     add     DWORD PTR [rbp-0x4], 0x1
0x000055555555275 <+93>:     mov     eax, DWORD PTR [rbp-0x4]
0x000055555555278 <+96>:     cmp     eax, DWORD PTR [rbp-0x14]
0x00005555555527b <+99>:     jle     0x555555555249 <simpleCounter+49>
0x00005555555527d <+101>:    mov     edi, 0xa
0x000055555555282 <+106>:    call    0x555555555090 <putchar@plt>
0x000055555555287 <+111>:    mov     eax, DWORD PTR [rip+0xddb]          # 0x5555555556068 <VAR>
0x00005555555528d <+117>:    mov     esi, eax
0x00005555555528f <+119>:    lea     rdi, [rip+0xdcc]                    # 0x5555555556062
0x000055555555296 <+126>:    mov     eax, 0x0
0x00005555555529b <+131>:    call    0x5555555550c0 <printf@plt>
0x0000555555552a0 <+136>:    nop
0x0000555555552a1 <+137>:    leave
0x0000555555552a2 <+138>:    ret
End of assembler dump.

```

7. How many print functions are in this current function you are stopped at?
Hint: There is more than one type of print statement.

- There are 4 print functions. There are 3 printf, and one putchar function.

8. Examining the disassembled code, grab the memory address of the constant from the function you're stopped at and display its location in the process image.

- The location of the constant is 0x555555556038

```

0x00005555555522f <+23>:      lea     rdi, [rip+0xe02]          # 0x5555555556038
0x000055555555236 <+30>:      mov     eax, 0x0
0x00005555555523b <+35>:      call    0x5555555550c0 <printf@plt>

```

9. Continue running (enter 15). What function are you currently stopped at, what is the name and the value being passed into the function you are currently stopped at?

- The function that we stopped in is simpleIf, and the value being passed in is num=15, which is what we entered in the terminal.

10. What is the value of the first string in this function you are stopped at, and what is the name and the type of the global variable in the function where you are stopped in?

Hint: notice the legend.

- The value is: “Sorry you didn’t win”, which is at 0x555555556019.

```
0x555555551ec <simpleIf+35> lea rdi, [rip + 0xe26] RDI => 0x555555556019 ← "Sorry you didn't win\n"
```

- The type of global variable in the function is a double named ‘MAX’.

```
0x555555551f8 <simpleIf+47> mov rax, qword ptr [rip + 0xe21] RAX, [MAX]
```

- What section (from the legend) is the global variable located in?

- It is located in the DATA section:

```
pwndbg> vmmap &MAX
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
Start      End Perm  Size Offset File
0x55555557000 0x55555558000 r--p    1000 2000 /home/ccrutter/lab3/lab3_p1
0x55555558000 0x55555559000 rw-p    1000 3000 /home/ccrutter/lab3/lab3_p1 +0x20
0x55555559000 0x5555555a000 rw-p   21000 0 [heap]
```

- Disassemble this function you are stopped at and capture the output

```
pwndbg> disasm simpleIf
Dump of assembler code for function simpleIf:
0x00005555551c9 <+0>: endbr64
0x00005555551cd <+4>: push rbp
0x00005555551ce <+5>: mov rbp, rsp
0x00005555551d1 <+8>: sub rsp, 0x10
0x00005555551d5 <+12>: mov DWORD PTR [rbp-0x4], edi
=> 0x00005555551d8 <+15>: cmp DWORD PTR [rbp-0x4], 0xc
0x00005555551dc <+19>: jne 0x555555551ec <simpleIf+35>
0x00005555551de <+21>: lea rdi, [rip+0xe23] # 0x555555556008
0x00005555551e5 <+28>: call 0x555555550a0 <puts@plt>
0x00005555551ea <+33>: jmp 0x555555551f8 <simpleIf+47>
0x00005555551ec <+35>: lea rdi, [rip+0xe26] # 0x555555556019
0x00005555551f3 <+42>: call 0x555555550a0 <puts@plt>
0x00005555551f8 <+47>: mov rax, QWORD PTR [rip+0xe21] # 0x555555558020 <MAX>
0x00005555551ff <+54>: movq xmm0, rax
0x0000555555204 <+59>: lea rdi, [rip+0xe24] # 0x55555555602f
0x000055555520b <+66>: mov eax, 0x1
0x0000555555210 <+71>: call 0x555555550c0 <printf@plt>
0x0000555555215 <+76>: nop
0x0000555555216 <+77>: leave
0x0000555555217 <+78>: ret
End of assembler dump.
```

- Look for the cmp instruction in the assembly from the disassembled code. What value is the variable from question 14 being compared to?

- It is being compared to 0xc, which is 12 in decimal.

```
cmp DWORD PTR [rbp-0x4], 0xc
```

- What area of the process image is the variable s located, what storage class is it? Hint: notice the value of s as it enters the function that is called twice.

```
0x55555558000 0x55555559000 rw-p 1000 3000 /home/ccrutter/lab3/lab3_p1 +0x10
```

- The variable s is located on the stack and has automatic storage duration.

- Display 64 addresses of the stack (NOTE: this is not backtrace).

```

pwndbg> stack 64
00:0000 rsp 0x7fffffff1e0 → 0x7fffffff328 → 0x7fffffff5ac ← '/home/ccrutter/lab3/lab3_p1'
01:0008 -018 0x7fffffff1e8 ← 0x1f7fe5af0
02:0010 -010 0x7fffffff1f0 → 0x7fffffff2e0 → 0x555555550e0 (_start) ← endbr64
03:0018 -008 0x7fffffff1f8 → 0x7fffffff328 → 0x7fffffff5ac ← '/home/ccrutter/lab3/lab3_p1'
04:0020 rbp 0x7fffffff200 → 0x7fffffff2a0 → 0x7fffffff300 ← 0
05:0028 +008 0x7fffffff208 → 0x7ffff7c2a1ca (_libc_start_call_main+122) ← mov edi, eax
06:0030 +010 0x7fffffff210 → 0x7fffffff250 ← 0
07:0038 +018 0x7fffffff218 → 0x7fffffff328 → 0x7fffffff5ac ← '/home/ccrutter/lab3/lab3_p1'
08:0040 +020 0x7fffffff220 ← 0x155554040
09:0048 +028 0x7fffffff228 → 0x555555552a3 (main) ← endbr64
0a:0050 +030 0x7fffffff230 → 0x7fffffff328 → 0x7fffffff5ac ← '/home/ccrutter/lab3/lab3_p1'
0b:0058 +038 0x7fffffff238 ← 0xf035a7b560c1a439
0c:0060 +040 0x7fffffff240 ← 1
0d:0068 +048 0x7fffffff248 ← 0
0e:0070 +050 0x7fffffff250 ← 0
0f:0078 +058 0x7fffffff258 → 0x7ffff7ffd000 (_rtld_global) → 0x7ffff7ffe2e0 → 0x555555554000 ← 0x10102464c457f
10:0080 +060 0x7fffffff260 ← 0xf035a7b561a1a439
11:0088 +068 0x7fffffff268 ← 0xf035b7cfe683a439
12:0090 +070 0x7fffffff270 ← 0x7fff00000000
13:0098 +078 0x7fffffff278 ← 0
14:00a0 +080 0x7fffffff280 ← 0
15:00a8 +088 0x7fffffff288 ← 1
16:00b0 +090 0x7fffffff290 ← 0
17:00b8 +098 0x7fffffff298 ← 0x2f0eb401dc527d00
18:00c0 +0a0 0x7fffffff2a0 → 0x7fffffff300 ← 0
19:00c8 +0a8 0x7fffffff2a8 → 0x7ffff7c2a28b (_libc_start_main+139) ← mov r15, qword ptr [rip + 0x1d8cf6]
1a:00d0 +0b0 0x7fffffff2b0 → 0x7fffffff338 → 0x7fffffff5c9 ← 'SHELL=/bin/bash'
1b:00d8 +0b8 0x7fffffff2b8 → 0x7ffff7ffe2e0 → 0x555555554000 ← 0x10102464c457f
1c:00e0 +0c0 0x7fffffff2c0 ← 0x7fff00000000
1d:00e8 +0c8 0x7fffffff2c8 → 0x555555552a3 (main) ← endbr64
1e:00f0 +0d0 0x7fffffff2d0 ← 0
1f:00f8 +0d8 0x7fffffff2d8 ← 0
20:0100 +0e0 0x7fffffff2e0 → 0x555555550e0 (_start) ← endbr64
21:0108 +0e8 0x7fffffff2e8 → 0x7fffffff320 ← 1
22:0110 +0f0 0x7fffffff2f0 ← 0
...
25:0128 +108 0x7fffffff308 → 0x5555555510e (_start+46) ← hlt
26:0130 +110 0x7fffffff310 → 0x7fffffff318 ← 0x38 /* '8' */
27:0138 +118 0x7fffffff318 ← 0x38 /* '8' */
28:0140 +120 0x7fffffff320 ← 1
29:0148 rsi 0x7fffffff328 → 0x7fffffff5ac ← '/home/ccrutter/lab3/lab3_p1'
2a:0150 +130 0x7fffffff330 ← 0
2b:0158 rdx 0x7fffffff338 → 0x7fffffff5c9 ← 'SHELL=/bin/bash'
2c:0160 +140 0x7fffffff340 → 0x7fffffff5d9 ← 'PWD=/home/ccrutter/lab3'
2d:0168 +148 0x7fffffff348 → 0x7fffffff5f2 ← 'LOGNAME=ccrutter'
2e:0170 +150 0x7fffffff350 → 0x7fffffff604 ← 'XDG_SESSION_TYPE=tty'
2f:0178 +158 0x7fffffff358 → 0x7fffffff619 ← '._=/usr/bin/pwndbg'
30:0180 +160 0x7fffffff360 → 0x7fffffff62b ← 'LINES=18'
31:0188 +168 0x7fffffff368 → 0x7fffffff634 ← 'HOME=/home/ccrutter'
32:0190 +170 0x7fffffff370 → 0x7fffffff649 ← 'LANG=en_US.UTF-8'
33:0198 +178 0x7fffffff378 → 0x7fffffff65a ← 0x524f4cd4f35f534c ('LS_COLOR')
34:01a0 +180 0x7fffffff380 → 0x7fffffff673 ← 'COLUMNS=60'
35:01a8 +188 0x7fffffff388 → 0x7fffffff67e ← 'SSH_CONNECTION=172.16.2.140 50257 10.202.12.75 22'
36:01b0 +190 0x7fffffff390 → 0x7fffffff6b0 ← 'LESSCLOSE=/usr/bin/lesspipe %s %s'
37:01b8 +198 0x7fffffff398 → 0x7fffffff6d2 ← 'XDG_SESSION_CLASS=user'
38:01c0 +1a0 0x7fffffff3a0 → 0x7fffffff6e9 ← 'TERM=xterm-256color'
39:01c8 +1a8 0x7fffffff3a8 → 0x7fffffff6fd ← 'PYTHONHOME=/usr/lib/pwndbg'
3a:01d0 +1b0 0x7fffffff3b0 → 0x7fffffff6e18 ← 'LESSOPEN=| /usr/bin/lesspipe %s'
3b:01d8 +1b8 0x7fffffff3b8 → 0x7fffffff6e38 ← 'USER=ccrutter'
3c:01e0 +1c0 0x7fffffff3c0 → 0x7fffffff6e47 ← 0x313d4c564c4853 /* 'SHLVL=1' */
3d:01e8 +1c8 0x7fffffff3c8 → 0x7fffffff6e4f ← 'XDG_SESSION_ID=3722'
3e:01f0 +1d0 0x7fffffff3d0 → 0x7fffffff6e63 ← 'XDG_RUNTIME_DIR=/run/user/1001'
3f:01f8 +1d8 0x7fffffff3d8 → 0x7fffffff6e82 ← 'SSH_CLIENT=172.16.2.140 50257 22'
pwndbg>

```

Part 2

1	Variable	Memory	Location	
2	main			High memory
3	array[5]	0x7ffdb770234	stack	
4	int array	0x7ffdb770220	stack	
5	ptr	0x7ffdb770218	stack	
6	char *word	0x7ffdb770210	stack	
7	int *ptr2	0x7ffdb770208	stack	
8	double *dptr2	0x7ffdb770200	stack	
9	double val2	0x7ffdb7701f8	stack	
10	double *dptr	0x7ffdb7701f0	stack	
11	double val	0x7ffdb7701e8	stack	
12	int y	0x7ffdb7701e4	stack	
13	int x	0x7ffdb7701e0	stack	
14				
15	-----	64 byte gap-----		
16	foo			
17	int a	0x7ffdb7701a0	stack	
18	int *b	0x7ffdb770198	stack	
19	int *c	0x7ffdb770190	stack	
20	double d	0x7ffdb770188	stack	
21	int i	0x7ffdb770184	stack	
22	int x	0x7ffdb770180	stack	
23	heap gap			
24	b	0x55e8969576f0	heap	
25	c	0x55e8969576d0	heap	Low memory
26	word	0x55e8969572a0		